ECE 220 Computer Systems & Programming

Lecture 21 — Intro to C++: Inheritance and Polymorphism

November 10, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

Announcements

- **Midterm 2 is graded** — median 83.0, mean 80.38

swap, Three Ways: swap3.cpp

```
void swap_val(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

void swap_ptr(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void swap_ref(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

```
int main() {
    int x = 2, y = 3;
    swap_val(x, y);
    cout << "val: " << x << " " << y << endl;
    swap_ptr(&x, &y);
    cout << "ptr: " << x << " " << y << endl;
    swap_ref(x, y);
    cout << "ref: " << x << " " << y << endl;
    return 0;
}
```

swap, Three Ways: `swap3.cpp`

```
void swap_val(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

void swap_ptr(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void swap_ref(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

```
int main() {
    int x = 2, y = 3;
    swap_val(x, y);
    cout << "val: " << x << " " << y << endl;
    swap_ptr(&x, &y);
    cout << "ptr: " << x << " " << y << endl;
    swap_ref(x, y);
    cout << "ref: " << x << " " << y << endl;
    return 0;
}
```

It prints

```
val:  2 3     ptr:  3 2     ref:  2 3
```

`swap_ref` has no `*` and no `&` in its body, and it swapped `main`'s `x` and `y` back.

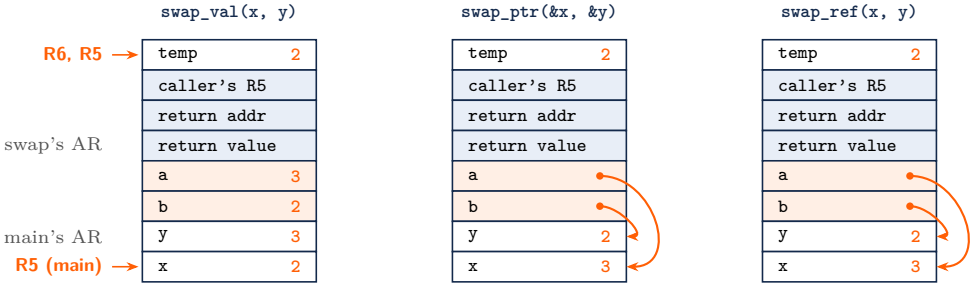
Today's Three Questions

A Rectangle is a Shape: it inherits Shape's members, and a `Shape *` can point at it.

1. `swap_ref` has no `*` and no `&` in its body. What does its activation record (AR) hold?
2. `class Rectangle : public Shape`. Which members does `Rectangle` get, and who may use them?
3. `ptr` is a `Shape *` and points at a `Rectangle`. Which `area()` does `ptr->area()` run?

What Each Call Puts on the Run-Time Stack

Each call enters with `x = 2`, `y = 3`. Draw its AR on the run-time stack (RTS) just before it returns.



- `swap_val`: `a` and `b` hold **copies of x and y**. `main`'s `x` and `y` never move
- `swap_ptr`: `a` and `b` hold **the addresses of x and y**; `*a` reaches `x`
- `swap_ref`: `a` and `b` hold **the addresses of x and y**: the AR of swap_ptr. The compiler writes the `*`: `a = b` in swap_ref compiles to the instructions of `*a = *b`.

A Reference Is a Second Name: `ref.cpp`

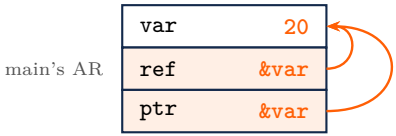
```cpp
int main() {
    int var = 5;
    int &ref = var; // permanent alias of var
    int *ptr = &var; // re-pointable

    ref = 10;
    cout << "var = " << var << endl;
    var = 15;
    cout << "ref = " << ref << endl;
    *ptr = 20;
    cout << "ref = " << ref << endl;
    cout << (&ref == &var) << endl;
    return 0;
}
```

What `ref.cpp` Puts on the Run-Time Stack

```
int main() {
    int var = 5;
    int &ref = var; // permanent alias of var
    int *ptr = &var; // re-pointable

    ref = 10;
    cout << "var = " << var << endl;
    var = 15;
    cout << "ref = " << ref << endl;
    *ptr = 20;
    cout << "ref = " << ref << endl;
    cout << (&ref == &var) << endl;
    return 0;
}
```



- `int &ref = var;` declares a **reference**: `ref` is another name for `var`, the same word of memory
- A reference is bound where it is declared and stays bound. A pointer can be re-pointed: `ptr = &other;`
- No `*`: use `ref` as you use `var`. A reference to an object reaches a member with `.`, a pointer with `->`
- There is no NULL reference and no reference arithmetic
- The program prints `var = 10`, `ref = 15`, `ref = 20` and `1`

A reference parameter costs one address on the stack and no copy. What a copy of an object costs, next.

The Copy Constructor and `const` References

```
class vector {
  protected:
    double angle_, length_;
  public:
    vector(double a, double l) {
        angle_ = a; length_ = l;
    }
    // copy constructor
    vector(const vector &obj) {
        angle_ = obj.angle_;
        length_ = obj.length_;
        std::cout << "copy" << std::endl;
    }
    double getLength() const { return length_; }
};
```

```
double len_val(vector v) { return v.getLength(); }
double len_ref(const vector &v) {
    return v.getLength();
}
```

```
int main() {
    vector a(1.5, 2);
    vector b = a;
    std::cout << b.getLength() << std::endl;
    std::cout << len_val(a) << std::endl;
    std::cout << len_ref(a) << std::endl;
    return 0;
}
```

It prints **copy 2 copy 2 2**

- `vector b = a;` and `len_val(a)` run the **copy constructor**: it builds a new object from an existing one, here **16 bytes**. Its parameter is a reference; a `vector` parameter would need a copy first.
- `const vector &v`: one address on the stack, no copy, and `len_ref` may not **change v**
- Write none and the compiler supplies one that copies **every member**
- Mark `const` every member function that changes nothing: otherwise `len_ref` **cannot call it**

Inheritance: Shape, Rectangle, Triangle

```
class Shape {
  protected:
    double width_, height_;
  public:
    Shape() { width_ = 0; height_ = 0; }
    Shape(double a, double b) {
        width_ = a; height_ = b;
    }
    double area() { return 0; }   // unknown shape
};
```

```
class Rectangle : public Shape {
  public:
    Rectangle(double a, double b) : Shape(a, b) {}
};
```

This `Rectangle` replaces Lecture 20's standalone `Rectangle`: the same two members and `area()`, now shared with `Triangle` through `Shape`.

- **Inheritance** defines a class from an existing class. The existing one is the **base class** (`Shape`); the new one is the **derived class** (`Rectangle`)
- `Rectangle` inherits `width_`, `height_` and `area()`. It adds a constructor and its own `area()`
- Not inherited: the base class's constructors, destructor, copy constructor, overloaded operators and friends
- `: Shape(a, b)` is the **member initializer list**: the base constructor runs **first**, then `Rectangle`'s body `{}`
- `Rectangle rec(3, 5)` holds **two doubles, 3 and 5**, like a `Shape`

Who May Use a Member: `orthovector`

```
// along an axis: 0 R, 1 U, 2 L, 3 D
class orthovector : public vector {
  protected:
    int d_;
  public:
    orthovector(int dir, double l) {
        const double halfPI = 1.570796327;
        d_ = dir;
        angle_ = dir * halfPI;   // protected
        length_ = l;
    }
    double hypotenuse(const orthovector &b) {
        if ((d_ + b.d_) % 2 == 0)
            return length_ + b.length_;
        return sqrt(length_ * length_
                    + b.length_ * b.length_);
    }
};
```

An `orthovector` is a `vector` along one of four directions.

used in	`public`	`protected`	`private`
own class	Y	Y	Y
derived class	Y	Y	N
outside (`main`)	Y	N	N

- `vector` makes `angle_` and `length_` `protected` so that **orthovector's constructor can set them**
- Were they `private`, `angle_ = dir * halfPI` would **not compile**; the two doubles would still be inside every `orthovector`
- `f.hypotenuse(h)` prints **5**

In-Class Problem: `area` for Rectangle and Triangle

Write the body of `area()` in each derived class, from the `width_` and `height_` it inherits from `Shape`, so that `main` prints the two areas.

```
int main() {
    Rectangle rec(3, 5);
    Triangle tri(4, 5);
    cout << "rec " << rec.area() << endl;
    cout << "tri " << tri.area() << endl;
    return 0;
```

```
class Rectangle : public Shape {
  public:
    Rectangle(double a, double b) : Shape(a, b) {}
    double area() {
        // width_ by height_
        return width_ * height_;
    }
};
```

```
class Triangle : public Shape {
  public:
    Triangle(double a, double b) : Shape(a, b) {}
    double area() {
        // base width_, height height_
        return width_ * height_ / 2;
    }
};
```

- `rec.area()` prints 15 and `tri.area()` prints 10
- The compiler picks `Rectangle::area` because rec's declared type is Rectangle
- Delete `Triangle`'s `area()` and `tri.area()` runs `Shape::area`, printing 0

The same two objects, with a `Shape *` in front of them, next.

Declared Type vs Actual Type: shape_ptr.cpp

```
int main() {
    Shape *ptr;
    Rectangle rec(3, 5);
    Triangle tri(4, 5);
    ptr = &rec;         // points to rec
    cout << "rec " << ptr->area() << endl;
    ptr = &tri;         // points to tri
    cout << "tri " << ptr->area() << endl;
    return 0;
}
```

The classes are those of the last slide. Predict the output:

rec 0 tri 0

- The **declared type** of `ptr` is what its declaration says: `Shape *`
- The **actual type** of `*ptr` is the class of the object it points to: first **Rectangle**, then **Triangle**
- `ptr = &rec` is legal: a `Rectangle` is a `Shape`; its first 16 bytes are a `Shape`
- `Shape` would need `virtual double area() { return 0; }`. Without it the compiler chooses `area` by the **declared type**, at **compile time**: **static binding**

Both calls run `Shape::area`. The pointer reaches the right object and the wrong function.

Polymorphism: one call, `ptr->area()`, runs a different function depending on the object. One keyword gives it, next.

`virtual`: the Object Decides

```
class Shape {
  protected:
    double width_, height_;
  public:
    Shape() { width_ = 0; height_ = 0; }
    Shape(double a, double b) {
        width_ = a; height_ = b;
    }
    virtual double area() { return 0; }
};
```

One word added to `Shape`; `Rectangle`, `Triangle` and `main` unchanged. Now it prints

```
rec 15      tri 10
```

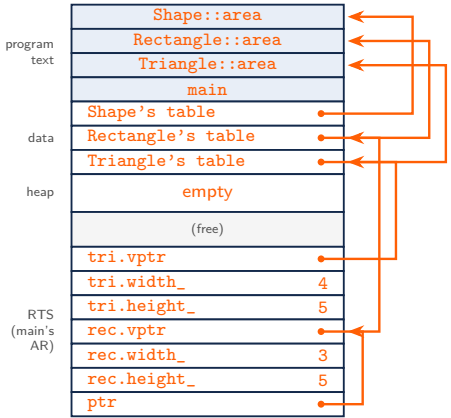
- A **virtual function** is a member function of the base class that derived classes are expected to redefine
- A derived class **overrides** it with a function of the same name and parameters. `virtual` is written once, in the base; the overrides are virtual too
- A call through a pointer or reference is bound at **run time**, by the **actual type**: **dynamic binding**
- `rec.area()` on the object itself: no change, the actual type is known at compile time

How does `ptr->area()` find `Rectangle::area` at run time? A table, next.

The Virtual Function Table

- Each class with a virtual function gets one **virtual function table** (vtbl): the addresses of its virtual functions. Here three tables, one entry each
- Each object of such a class carries one hidden pointer to its class's table, stored by the **constructor**
- `vsize.cpp` on this Mac (arm64, 8-byte pointers): `sizeof` of a `Shape` without `virtual` is **16**; with it, **24**
- `ptr->area()`: load the table pointer from `*ptr`, load entry 0 (fixed at compile time), call it. In LC-3: `LDR`, `LDR`, `JSRR`

Fill in the picture for `shape_virtual.cpp` after `ptr = &rec`: the `area` functions, the tables, and `main`'s `ptr`, `rec`, `tri`.



Pure Virtual Functions and Abstract Classes

```
class Shape {
  protected:
    double width_, height_;
  public:
    Shape(double a, double b) {
        width_ = a; height_ = b;
    }
    virtual double area() = 0;   // pure virtual
};
```

```
int main() {
    // Shape shape1(2, 4);   // error, see below
    Shape *ptr;              // allowed
    Rectangle rec(3, 5);
    ptr = &rec;
    cout << ptr->area() << endl;
    return 0;
}
```

Remove the `//` before `Shape shape1(2, 4);` and g++ reports:

```
abstract.cpp:22:11: error: variable type
'Shape' is an abstract class
abstract.cpp:12:20: note: unimplemented pure
virtual method 'area' in 'Shape'
```

- `= 0` makes `area` a **pure virtual function**: declared in `Shape`, with no body
- A class with a pure virtual function is an **abstract class**. No object of it can be created; pointers and references to it are allowed
- A derived class that does not define `area` is **abstract too**
- `Shape` is now an **interface**: every object reached through a `Shape *` has an `area()`

In-Class Problem: One Function for Every Shape

```
class Square : public Rectangle {
  public:
    Square(double s) : Rectangle(s, s) {}
};
```

```
int main() {
    Rectangle rec(3, 5);
    Triangle tri(4, 5);
    Square sq(2);
    Shape *list[3] = {&rec, &tri, &sq};
    cout << total_area(list, 3) << endl;
    return 0;
}
```

`Shape`, `Rectangle` and `Triangle` are those of `shape_virtual.cpp`, with `area() = 0` in `Shape`.

```
double total_area(Shape *s[], int n) {
    // add up the n areas
    double total = 0;
    for (int i = 0; i < n; i++)
        total += s[i]->area();
    return total;
}
```

- `main` prints **29**
- `Square` defines no `area`; `sq.area()` runs `Rectangle::area`, inherited
- `s[i]->area()` uses the table of **the object `s[i]` points to**
- A fourth shape class needs **no change** to `total_area`

Summary & What's Next

Today

- **References**: `int &a` is a second name for the argument. On the RTS it is an address; the compiler writes the `*`
- **Copy constructor**: runs for `vector b = a` and for every object passed by value. `const vector &` passes an address and forbids changes
- **Inheritance**: `class Rectangle : public Shape` gets `Shape`'s members; `protected` opens a member to derived classes; `: Shape(a, b)` runs the base constructor first
- **Polymorphism**: through a `Shape *`, a `virtual` function is chosen by the actual type at run time, through the object's hidden pointer to its class's virtual function table
- **Abstract classes**: `area() = 0` is pure virtual; `Shape` has no objects and serves as an interface

Next lecture (Thursday, November 12)

- Iterators and templates: `getmin<T>`, a class template, the library's `vector` and `list`, and the copy constructor again: shallow vs deep copy

Code: `swap3.cpp`, `ref.cpp`, `copyctor.cpp`, `shape.cpp`, `orthovector.cpp`, `shape_ptr.cpp`, `shape_virtual.cpp`, `vsize.cpp`, `abstract.cpp`, `total_area.cpp`, skeletons `shape_skel.cpp`, `total_area_skel.cpp`.